

Project 2 Report: Employee Attrition

Stacy Bowler
Utah State University
stacy.bowler@sdl.usu.edu

Derrik Miller
Utah State University
derrik.miller@sdl.usu.edu

2026-08-07

Abstract This project develops a blank.

Introduction

The objective of this project is to develop a predictive model capable of determining which employees are most likely to leave. This can allow the business to focus their retention efforts on high-risk employees.

The business decision addressed by this analysis is:

How much financial value does using this predictive model create compared with not using one?

The resulting model can be used to identify employees that appear likely to leave.

Business Objective

The primary objective is to accurately determine which employees are likely to quit before they do.

A practical utility function for this problem is:

$$U = \begin{cases} 0.60 \times 40000 - 8000 = 16000 & \text{if a retention intervention is provided and prevents attrition} \\ -8000 & \text{if a retention intervention is provided unnecessarily} \\ 0 & \text{if no intervention is provided and the employee leaves} \\ 0 & \text{if no intervention is provided and the employee stays} \end{cases}$$

The utility function reflects the incremental financial impact of acting on the model's recommendations. A true positive represents the expected net savings from successfully retaining an employee, while a false positive represents the cost of an unnecessary retention intervention. False negatives and true negatives are assigned zero utility because they represent the baseline outcome in which no intervention occurs.

assumptions:

- Cost of employee turnover: \$40,000
- Cost of retention intervention: \$8,000
- Retention intervention success rate: 60%

Plan

The analysis follows the regression modeling workflow:

1. Define the business problem.
2. Acquire and clean employee data.
3. Explore relationships between candidate predictors and attrition.
4. Build an interpretable regression model.
5. Evaluate assumptions and model fit.
6. Generate predictions and business recommendations.

The ideal dataset would contain:

- Age
- Attrition
- Salary
- Years at the company
- Overtime
- Job role
- Job satisfaction score

Due to the data being fake generated data, it was more than ideal. It included everything we thought of and much more.

Build

Data Source

The dataset was obtained from Kaggle at <https://www.kaggle.com/datasets/pavansubhasht/ibm-hr-analytics-attrition-dataset> and contains a fictional dataset created by IBM data scientists.

Variables initially available included 35 different point including notably:

- Age
- Attrition
- Department
- Gender
- Job Role
- Job Satisfaction
- Martial Status
- Monthly Income
- Number of Companies Worked
- Overtime

- Years Since Last Promotion
- Year at Company
- Years in Current Role

Data Cleaning

The following preprocessing steps were performed:

- Converted Attrition to a binary outcome where Yes = 1

```
import os
import numpy as np
import polars as pl
import pandas as pd
import seaborn.objects as so
import seaborn as sns
import matplotlib.pyplot as plt
import statsmodels.api as sm
import statsmodels.formula.api as smf

from patsy import dmatrices
from sklearn.base import clone
from sklearn.model_selection import train_test_split, StratifiedKFold
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import LogisticRegression
from sklearn.pipeline import Pipeline
from sklearn.metrics import (
    confusion_matrix,
    accuracy_score,
    precision_score,
    recall_score,
    roc_auc_score
)
from statsmodels.stats.outliers_influence import variance_inflation_factor
# from sklearn.utils import resample

pl.Config.set_tbl_rows(-1)
pl.Config.set_tbl_cols(-1)
pl.Config.set_fmt_str_lengths(100)

employee_data = (pl.read_csv(os.path.join('data', 'HR-Employee-Attrition.csv')))
# Make the outcome binary where 'Yes' = 1
.employee_data.with_columns(
    pl.when(pl.col('Attrition') == 'Yes').then(1)
    .when(pl.col('Attrition') == 'No').then(0)
```

```

        .alias('Left')
    )
    .select(pl.exclude(['Attrition', 'Department', 'EmployeeNumber', 'DailyRate',
        'MonthlyIncome', 'EmployeeCount', 'Over18', 'PerformanceRating']))
    )

```

Explore

Exploratory Data Analysis

Initial visualizations focused on the relationships between attrition and major numerical predictors.

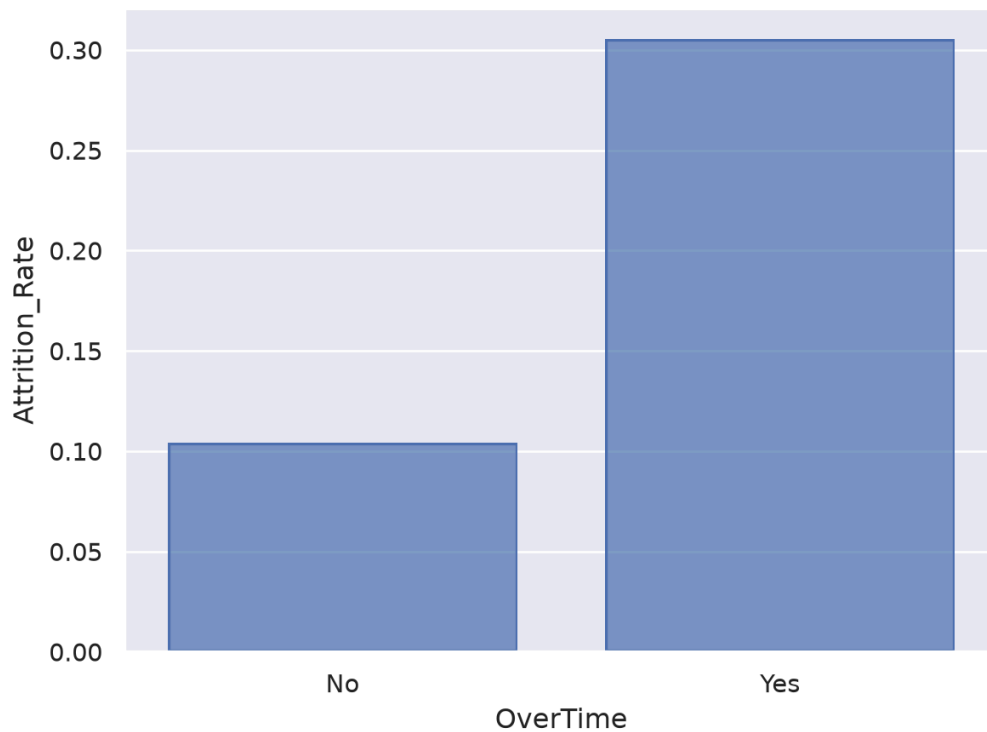
Overtime vs Attrition

```

overtime_rate = (
    employee_data
    .group_by("OverTime")
    .agg(Attrition_Rate=pl.col("Left").mean())
)

so.Plot(overtime_rate.to_pandas(),
        x="OverTime",
        y="Attrition_Rate") \
    .add(so.Bar())

```



This looks like employees that work overtime are more likely to leave.

Satisfaction vs Attrition

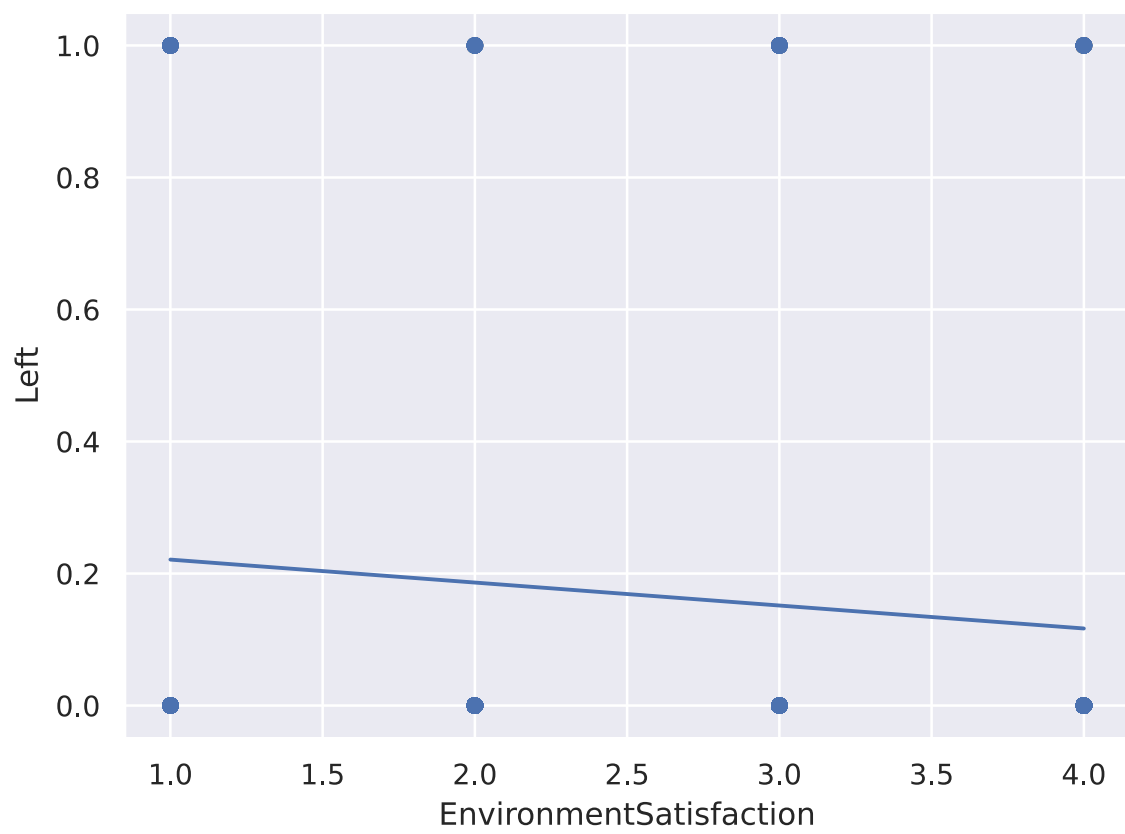
```
(
    so.Plot(employee_data.to_pandas(),
            x="JobSatisfaction",
            y="Left")
    .add(so.Dot(alpha=.20))
    .add(so.Line(), so.PolyFit(order=1))
).show()

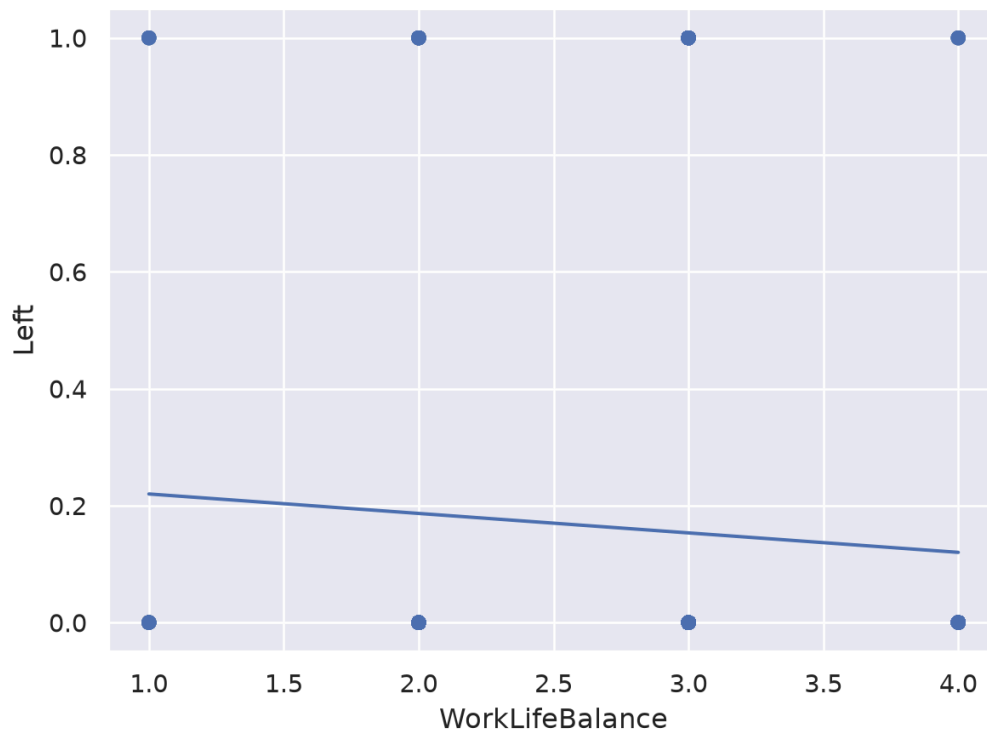
(
    so.Plot(employee_data.to_pandas(),
            x="EnvironmentSatisfaction",
            y="Left")
    .add(so.Dot(alpha=.20))
    .add(so.Line(), so.PolyFit(order=1))
).show()

(
    so.Plot(employee_data.to_pandas(),
            x="WorkLifeBalance",
```

```
        y="Left")  
    .add(so.Dot(alpha=.20))  
    .add(so.Line(), so.PolyFit(order=1))  
    )
```



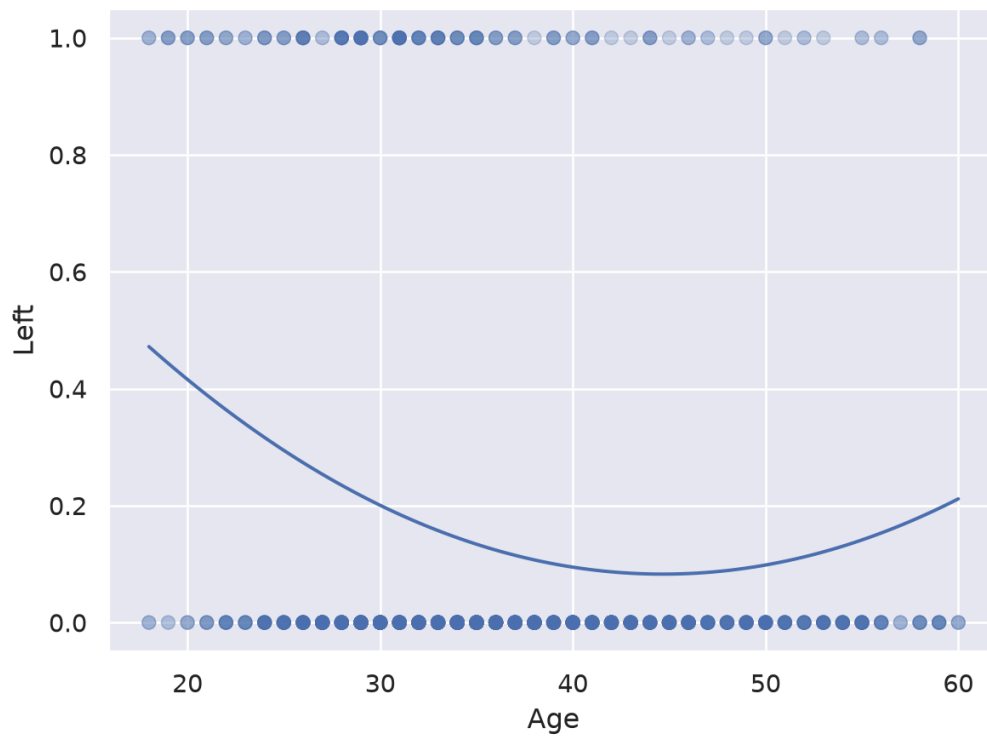




These graphs show what we would expect, that as employees report being more satisfied or having better work/life balance, they are less likely to leave.

Age vs Attrition

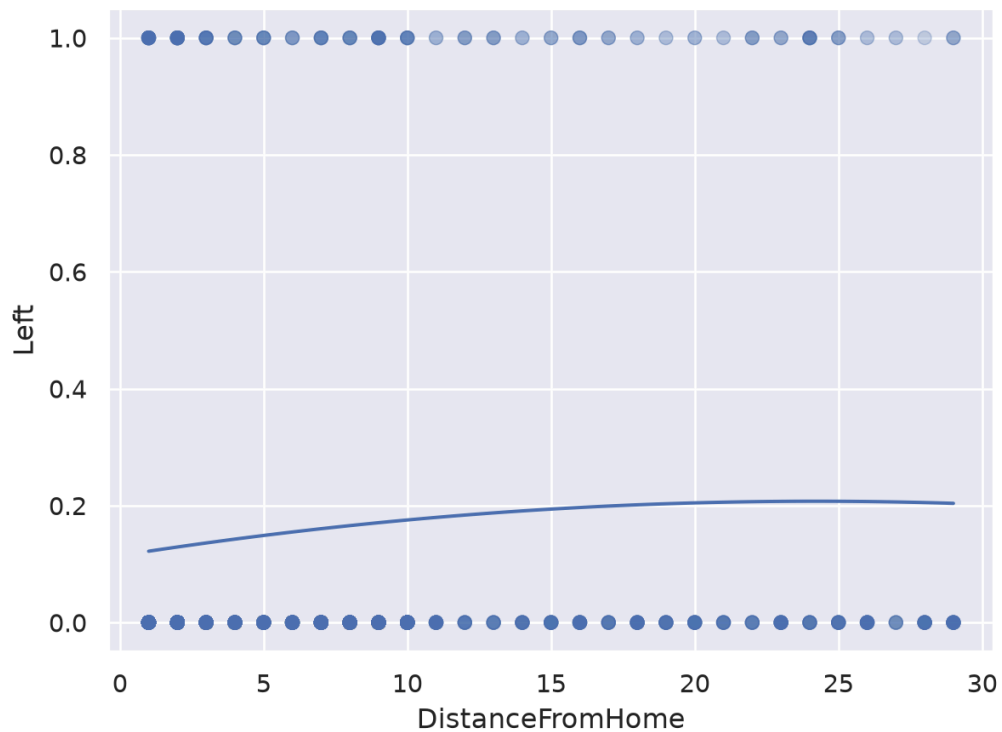
```
(  
    so.Plot(employee_data.to_pandas(),  
            x="Age",  
            y="Left")  
    .add(so.Dot(alpha=.15))  
    .add(so.Line(), so.PolyFit(order=2))  
)
```

We can see that younger employees are generally more likely to leave, but this starts to reverse around age 45. Retirement is likely a factor, but also employees are likely changing jobs for more senior roles.

Commute vs Attrition

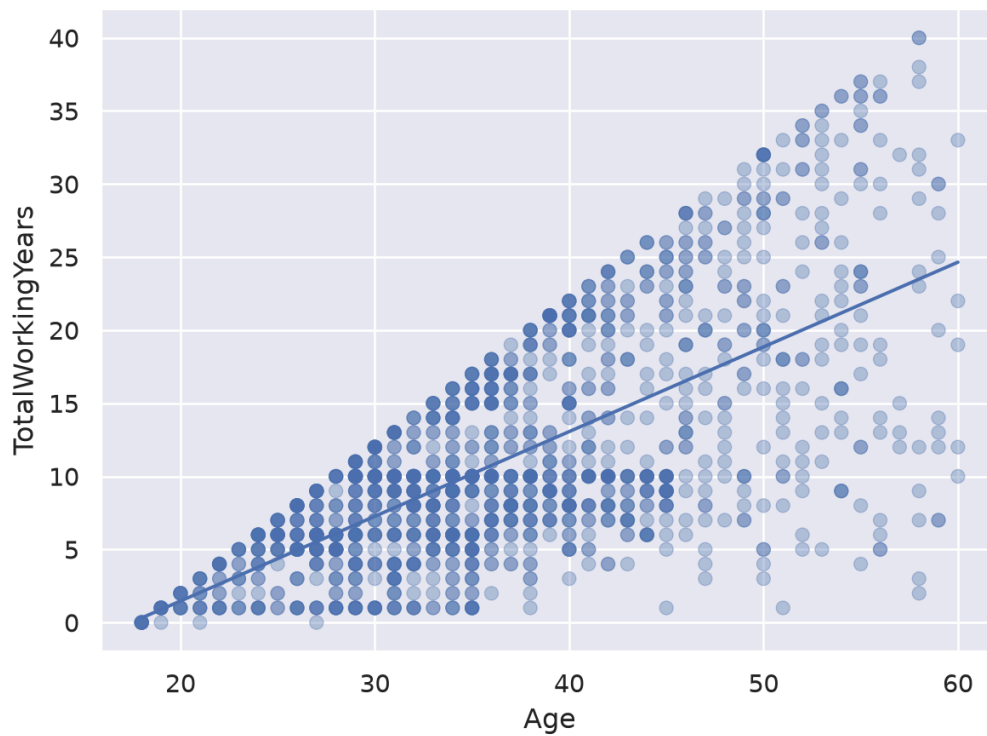
```
(
  so.Plot(employee_data.to_pandas(),
    x="DistanceFromHome",
    y="Left")
  .add(so.Dot(alpha=.15))
  .add(so.Line(), so.PolyFit(order=2))
)
```



There is a fairly weak correlation between how far someone lives from work and if they are likely to leave. It is interesting that the trend appears to level off so it seems like it's only a factor within a certain range.

Age vs Total Working Years

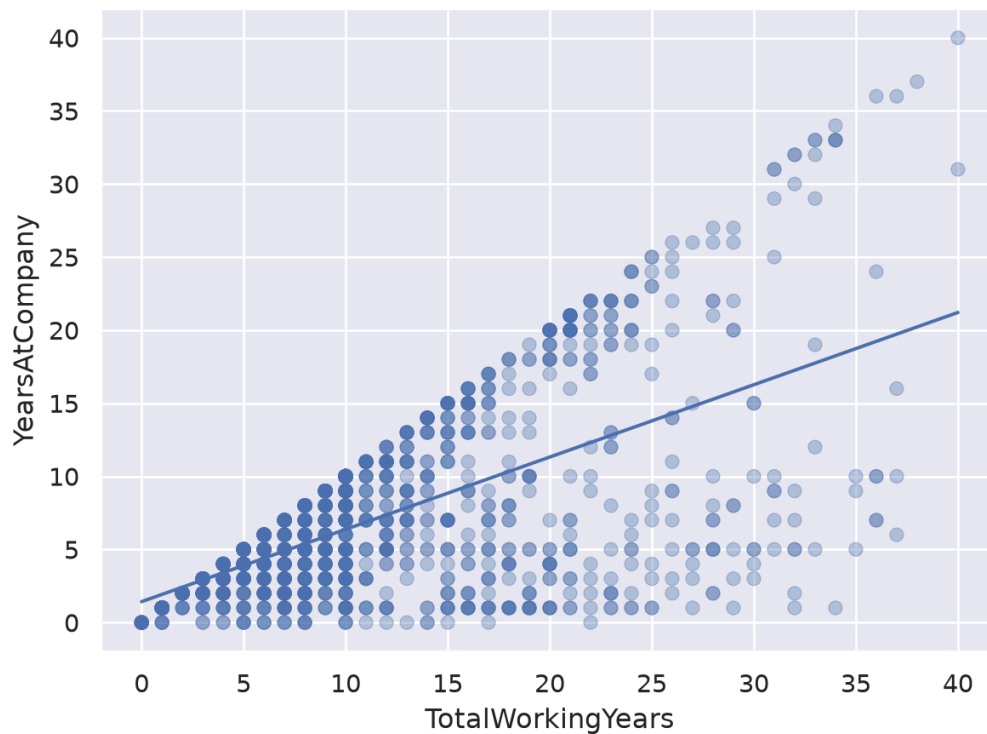
```
(
    so.Plot(employee_data.to_pandas(),
            x="Age",
            y="TotalWorkingYears")
    .add(so.Dot(alpha=.35))
    .add(so.Line(), so.PolyFit(order=1))
)
```



We thought there would be a strong relationship in the data that could cause issues between employee age and total working years. It looks like there is a relationship, but the correlation is not nearly as strong as we thought.

Years at Company vs Total Working Years

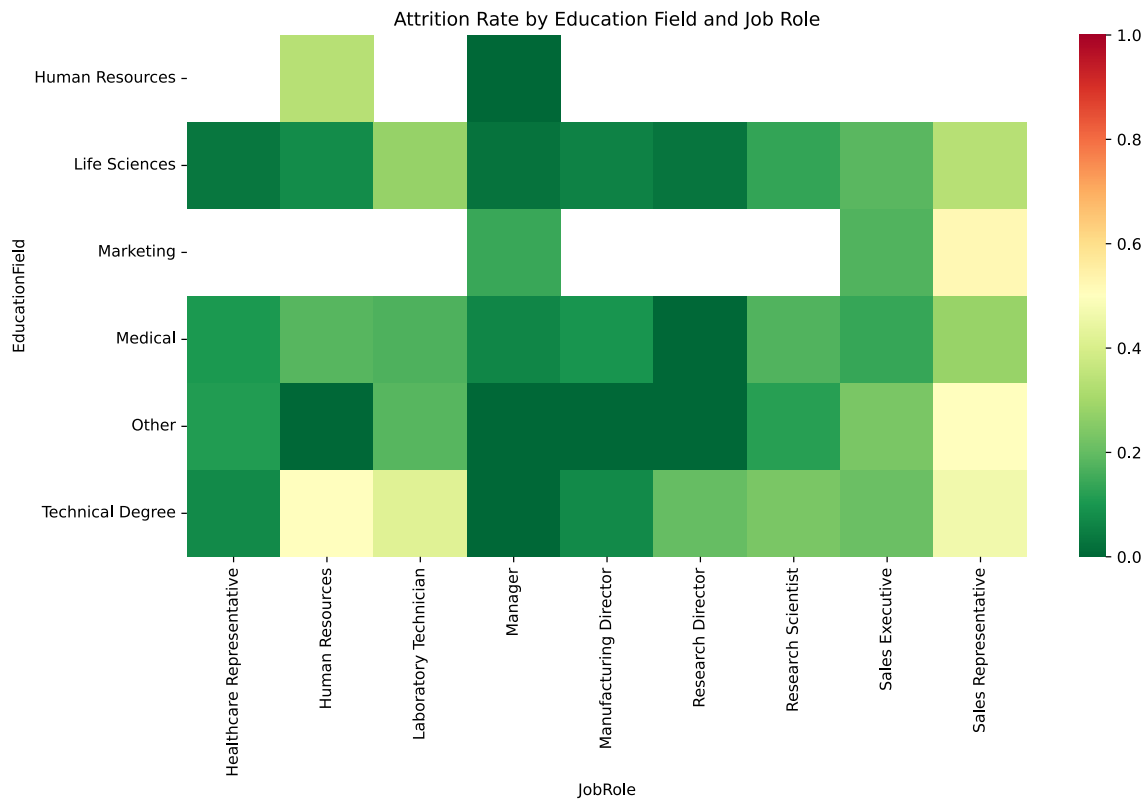
```
(  
    so.Plot(employee_data.to_pandas(),  
            x="TotalWorkingYears",  
            y="YearsAtCompany")  
    .add(so.Dot(alpha=.35))  
    .add(so.Line(), so.PolyFit(order=1))  
)
```



It looks like there is a relationship between years at the company and total working years, but not a very strong one.

Education Field vs Job Role

```
rate = (
    employee_data
    .group_by(["EducationField", "JobRole"])
    .agg(attrition_rate=pl.col("Left").mean())
    .to_pandas()
)
pivot_rate = rate.pivot(
    index="EducationField",
    columns="JobRole",
    values="attrition_rate"
)
plt.figure(figsize=(12,6))
sns.heatmap(pivot_rate, cmap="RdYlGn_r", vmin=0, vmax=1)
plt.title("Attrition Rate by Education Field and Job Role")
plt.show()
```

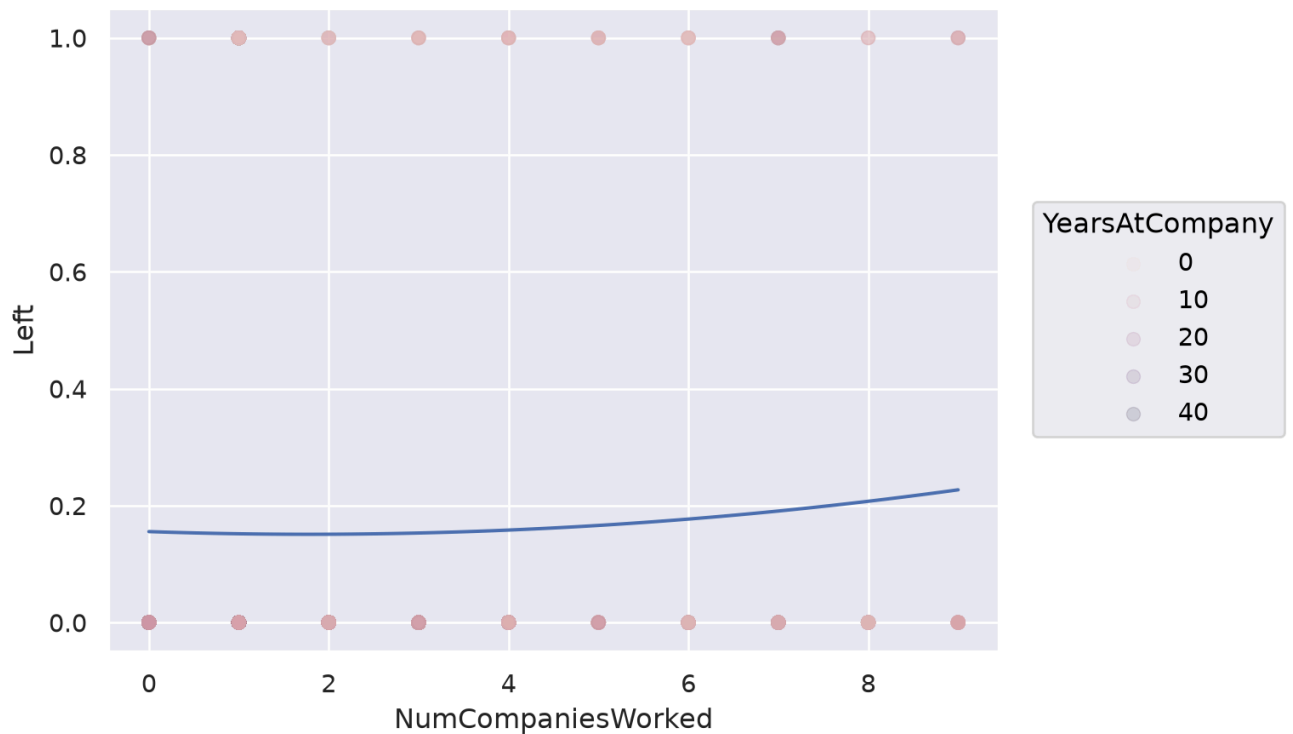


The heatmap reveals a strong structural relationship between education field and job role, indicating that employees are not randomly distributed across roles. Instead, specific education backgrounds concentrate within particular job families. This dependency explains multicollinearity we later observed in the initial logistic regression model.

Attrition varies more meaningfully across job roles than across education fields, suggesting that occupational role is a stronger predictor of turnover than educational background.

Number of Companies and Attrition

```
(
  so.Plot(employee_data.to_pandas(),
    x="NumCompaniesWorked",
    y="Left")
  .add(so.Dot(alpha=.15), color="YearsAtCompany")
  .add(so.Line(), so.PolyFit(order=2))
)
```



This is an interesting graph because it shows that employees that have worked are more companies are more likely to leave. This could be explained by the fact that employees that have worked at more companies are more likely to be looking for new opportunities. It also shows that employees that have been at the company for a long time are less likely to leave, which makes sense.

Key Findings

- Found this.
- Found that.

Reconcile

Sampling Strategy

A stratified sampling approach was used to evenly split test and training data with even attrition and non-attrition employees.

Assumptions

Model Refinements

Several iterations of the model were evaluated.

Adjustments included:

- abcd
- GDs.

- Evaluating Variance Inflation Factors (VIFs).

Hyper-parameter Tuning

```

employee_data = employee_data.to_dummies(
    columns=[
        'BusinessTravel',
        'EducationField',
        'Gender',
        'JobRole',
        'MaritalStatus',
        'OverTime'
    ]
).to_pandas()

# This is a utility function meant to maximize the use of money to retain employees
# It is based on our assumptions:
# * Cost of employee turnover: $40,000
# * Cost of retention intervention: $8,000
# * Retention intervention success rate: 60%
def utility(matrix):
    true_negative, false_positive, false_negative, true_positive = matrix.ravel()

    return (
        true_positive * 16000 +
        false_positive * (-8000) +
        false_negative * 0 +
        true_negative * 0
    )

X = employee_data.drop(columns="Left")
y = employee_data["Left"]

X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.20,
    random_state=42,
    stratify=y
)

# Candidate cutoff probabilities and c values
c_values = np.logspace(-2, 2, 15)
thresholds = np.arange(0.20, 0.61, 0.05)

```

```

models = {
    "Ridge": LogisticRegression(l1_ratio=0),
    "Lasso": LogisticRegression(
        l1_ratio=1,
        solver="saga",
        max_iter=5000,
        random_state=42
    ),
    "Elastic": LogisticRegression(
        solver="saga",
        l1_ratio=0.5,
        max_iter=5000,
    )
}

# Setup proper 5-fold cross-validation object
kFolds = StratifiedKFold(n_splits=5, shuffle=True, random_state=42)

results = []

# We will try all 3 different models to see which one seems the best
for model_name, base_model in models.items():

    # We are going to tune our cutoff probability and c value at the same time
    best = {
        "C": None,
        "Threshold": None,
        "Utility": -np.inf,
        "Accuracy": 0,
        "Precision": 0,
        "Recall": 0,
        "AUC": 0
    }

    # Loop through each regularization parameter (C)
    for C in c_values:
        for threshold in thresholds:
            fold_utilities = []
            fold_accuracy = []
            fold_precision = []
            fold_recall = []
            fold_auc = []

```



```

# Iterate through the 5 distinct validation folds
for train_idx, val_idx in kFolds.split(X_train, y_train):

    # Split using the index arrays generated by StratifiedKFold
    X_train_fold = X_train.iloc[train_idx]
    X_val_fold = X_train.iloc[val_idx]

    y_train_fold = y_train.iloc[train_idx]
    y_val_fold = y_train.iloc[val_idx]

    # grab our frequentist logistic regression model and set C
    model = Pipeline([
        ("scaler", StandardScaler()),
        ("logistic", clone(base_model).set_params(C=C))
    ])

    model.fit(X_train_fold, y_train_fold)

    probabilities = model.predict_proba(X_val_fold)[:,1]

    predictions = (probabilities >= threshold).astype(int)

    cm = confusion_matrix(y_val_fold, predictions)

    fold_utilities.append(utility(cm))
    fold_accuracy.append(accuracy_score(y_val_fold, predictions))
    fold_precision.append(
        precision_score(y_val_fold, predictions, zero_division=0)
    )
    fold_recall.append(
        recall_score(y_val_fold, predictions, zero_division=0)
    )

    # AUC should use probabilities, not binary predictions
    fold_auc.append(
        roc_auc_score(y_val_fold, probabilities)
    )

avg_utility = np.mean(fold_utilities)
avg_accuracy = np.mean(fold_accuracy)
avg_precision = np.mean(fold_precision)
avg_recall = np.mean(fold_recall)

```

```

avg_auc = np.mean(fold_auc)

if avg_utility > best["Utility"]:
    best["C"] = C
    best["Threshold"] = threshold
    best["Utility"] = avg_utility
    best["Accuracy"] = avg_accuracy
    best["Precision"] = avg_precision
    best["Recall"] = avg_recall
    best["AUC"] = avg_auc

# It's very possible we tie, if so go by accuracy
elif (
    avg_utility == best["Utility"]
    and avg_accuracy > best["Accuracy"]
):
    best["C"] = C
    best["Threshold"] = threshold
    best["Utility"] = avg_utility
    best["Accuracy"] = avg_accuracy
    best["Precision"] = avg_precision
    best["Recall"] = avg_recall
    best["AUC"] = avg_auc

# Save the best settings for this model
results.append({
    "Model": model_name,
    **best
})

# Identify the best cutoff probability, best C, and best model
results = pd.DataFrame(results)

results.sort_values(
    "Utility",
    ascending=False
).head()

```

	Model	C	Threshold	Utility	Accuracy	Precision	Recall	AUC
0	Ridge	0.071969	0.35	260800.0	0.885204	0.671458	0.568421	0.842234
2	Elastic	0.517947	0.40	260800.0	0.889452	0.701604	0.542105	0.841832
1	Lasso	1.000000	0.40	257600.0	0.887753	0.690919	0.542105	0.841591

Elastic Net achieved the highest cross-validated business utility (tied with Ridge), the highest overall accuracy, and the highest precision while maintaining an AUC comparable to the other models. Although Ridge produced slightly higher recall, the difference was small. Therefore, Elastic Net was selected as the final model because it provided the best overall balance of predictive performance and business value.

The similarity in performance suggests that the employee attrition dataset contains a relatively stable set of predictors with limited multicollinearity. Consequently, the choice of regularization method had only a modest impact on predictive performance. Our removal from the data of some predictors where we saw or predicted multicollinearity may have also contributed.

```
final_c = 0.517947
final_probability = 0.40

final_model = Pipeline([
    ("scaler", StandardScaler()),
    ("classification", LogisticRegression(
        solver="saga",
        l1_ratio=0.5,
        C=final_c,
        max_iter=5000,
        random_state=42
    ))
])

final_model.fit(X_train, y_train)

p_test = final_model.predict_proba(X_test)[: , 1]
y_pred = (p_test >= final_probability).astype(int)
final_accuracy = accuracy_score(y_test, y_pred)

final_accuracy

cm = confusion_matrix(y_test, y_pred)
print(utility(cm))

employee_data = employee_data.rename(columns={
    "BusinessTravel_Non-Travel": "BusinessTravel_NonTravel",
    "EducationField_Human Resources": "EducationField_HumanResources",
    "EducationField_Life Sciences": "EducationField_LifeSciences",
    "EducationField_Technical Degree": "EducationField_TechnicalDegree",
    "JobRole_Healthcare Representative": "JobRole_HealthcareRepresentative",
    "JobRole_Human Resources": "JobRole_HumanResources",
    "JobRole_Laboratory Technician": "JobRole_LaboratoryTechnician",
```

```

    "JobRole_Manufacturing Director": "JobRole_ManufacturingDirector",
    "JobRole_Research Director": "JobRole_ResearchDirector",
    "JobRole_Research Scientist": "JobRole_ResearchScientist",
    "JobRole_Sales Executive": "JobRole_SalesExecutive",
    "JobRole_Sales Representative": "JobRole_SalesRepresentative",
  })

predictors = [
  "Age",
  "BusinessTravel_NonTravel",
  "BusinessTravel_Travel_Frequently",
  "BusinessTravel_Travel_Rarely",
  # "C(Department)", # job role perfectly maps to certain departments except
manager
  "DistanceFromHome",
  "EducationField_HumanResources", "EducationField_LifeSciences",
  "EducationField_Marketing", "EducationField_Medical", "EducationField_Other",
  "EducationField_TechnicalDegree", # This seems to be redundant.
  "Education",
  "EnvironmentSatisfaction",
  "Gender_Female",
  "HourlyRate", # Hourly rate seems to group employees together better than other
similar rate columns that tend to only match one or few employees.
  "JobInvolvement",
  "JobLevel",
  "JobRole_HealthcareRepresentative", "JobRole_HumanResources",
  "JobRole_LaboratoryTechnician",
  "JobRole_Manager", "JobRole_ManufacturingDirector", "JobRole_ResearchDirector",
  "JobRole_ResearchScientist",
  "JobRole_SalesExecutive", "JobRole_SalesRepresentative",
  "JobSatisfaction",
  "MaritalStatus_Divorced", "MaritalStatus_Married", "MaritalStatus_Single",
  "MonthlyRate", # may be more linear and likely redundant with hourly rate
  "NumCompaniesWorked",
  "OverTime_Yes",
  "PercentSalaryHike",
  "RelationshipSatisfaction",
  "TotalWorkingYears", # may correlate strongly with Age
  "TrainingTimesLastYear",
  "WorkLifeBalance",
  "YearsAtCompany",
  "YearsInCurrentRole",
  "YearsSinceLastPromotion",

```

```

    "YearsWithCurrManager"
]

formula = "Left ~ " + " + ".join(predictors)

# fit_01 = smf.logit(
#     formula,
#     data=X_train
# ).fit()

# print(fit_01.summary())

# # Create the design matrix used by the model
# y, X = dmatrices(formula, data=pb_train, return_type="dataframe")

# # Calculate VIF
# vif = pd.DataFrame({
#     "Variable": X.columns,
#     "VIF": [variance_inflation_factor(X.values, i) for i in range(X.shape[1])]
# })

# print(vif.sort_values("VIF", ascending=False))

```

232000

Given the high P values, we should consider removing these from our model

Variable	p-value
Education 2–5	0.60–0.94
HourlyRate	0.656
PercentSalaryHike	0.584
YearsAtCompany	0.704

Then these next

Variable	p-value
JobLevel 3	0.936
JobLevel 4	0.528
JobLevel 5	0.466
JobRole Manager	0.846

Variable	p-value
JobRole Research Scientist	0.834
Manufacturing Director	0.441

Fit

Final Model

Bootstrap Confidence Intervals

```

from sklearn.utils import resample
from sklearn.linear_model import LogisticRegression

n_iterations = 1000
boot_coefs = []

X_train_clean = X_train.drop(columns=['Intercept'], errors='ignore')
X_test_clean = X_test.drop(columns=['Intercept'], errors='ignore')

scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train_clean)
X_test_scaled = scaler.transform(X_test_clean)

for i in range(n_iterations):
    # Resample training data
    X_boot, y_boot = resample(X_train_scaled, y_train, random_state=i)

    # Refit model with the fixed optimal hyperparameter
    boot_model = LogisticRegression(l1_ratio=0.5, solver='saga', C=final_c,
max_iter=5000, random_state=42)
    boot_model.fit(X_boot, y_boot)
    boot_coefs.append(boot_model.coef_.flatten())

boot_coefs = np.array(boot_coefs)

# Calculate 95% Percentile Confidence Intervals
lower_bounds = np.percentile(boot_coefs, 2.5, axis=0)
upper_bounds = np.percentile(boot_coefs, 97.5, axis=0)

ci_df = pd.DataFrame({
    'Feature': X_train_clean.columns,
    'Optimal_Coeff': boot_model.coef_.flatten(),
    '2.5%': lower_bounds,

```

```

    '97.5%': upper_bounds
})
print(ci_df)

```

	Feature	Optimal_Coef	2.5%	97.5%
0	Age	-0.363739	-0.728848	-0.082505
1	BusinessTravel_Non-Travel	-0.265356	-0.587555	-0.105176
2	BusinessTravel_Travel_Frequently	0.424046	0.181975	0.549378
3	BusinessTravel_Travel_Rarely	0.000000	0.000000	0.000000
4	DistanceFromHome	0.348508	0.188686	0.575625
5	Education	0.035551	-0.166596	0.202203
6	EducationField_Human Resources	0.103594	-0.013504	0.344272
7	EducationField_Life Sciences	0.000000	-0.151258	0.132224
8	EducationField_Marketing	0.189447	-0.080347	0.281892
9	EducationField_Medical	-0.049055	-0.297073	0.034224
10	EducationField_Other	-0.072893	-0.485287	0.000255
11	EducationField_Technical Degree	0.161404	0.000000	0.349063
12	EnvironmentSatisfaction	-0.442040	-0.703227	-0.279672
13	Gender_Female	-0.103835	-0.175213	0.019587
14	Gender_Male	0.103835	-0.019587	0.175213
15	HourlyRate	0.141777	-0.215775	0.167301
16	JobInvolvement	-0.343945	-0.578676	-0.189995
17	JobLevel	0.273450	-0.208473	0.705606
18	JobRole_Healthcare Representative	-0.203936	-0.606919	0.000000
19	JobRole_Human Resources	0.081555	-0.106171	0.257042
20	JobRole_Laboratory Technician	0.396873	0.177909	0.613267
21	JobRole_Manager	-0.200968	-0.417214	0.156988
22	JobRole_Manufacturing Director	-0.018044	-0.504230	0.011228
23	JobRole_Research Director	-0.395224	-0.930583	-0.107048
24	JobRole_Research Scientist	0.000000	-0.345913	0.023468
25	JobRole_Sales Executive	0.174694	0.000000	0.412139
26	JobRole_Sales Representative	0.320465	0.157587	0.509597
27	JobSatisfaction	-0.300835	-0.644665	-0.227989
28	MaritalStatus_Divorced	-0.118561	-0.286365	0.000000
29	MaritalStatus_Married	0.000000	-0.212382	0.000000
30	MaritalStatus_Single	0.458455	0.037700	0.598234
31	MonthlyRate	-0.023955	-0.171303	0.213903
32	NumCompaniesWorked	0.582944	0.272875	0.675616
33	Overtime_No	-0.451459	-0.540525	-0.331858
34	Overtime_Yes	0.451459	0.331858	0.540525
35	PercentSalaryHike	-0.039945	-0.274104	0.113741
36	RelationshipSatisfaction	-0.387828	-0.520161	-0.068507

37	StandardHours	0.000000	0.000000	0.000000
38	StockOptionLevel	-0.023451	-0.642062	0.033432
39	TotalWorkingYears	-0.707473	-1.012441	-0.026431
40	TrainingTimesLastYear	-0.186286	-0.388010	0.007209
41	WorkLifeBalance	-0.303536	-0.512531	-0.078343
42	YearsAtCompany	0.133094	-0.392533	0.591344
43	YearsInCurrentRole	-0.313912	-0.705821	-0.032934
44	YearsSinceLastPromotion	0.471753	0.243899	0.884166
45	YearsWithCurrManager	-0.423168	-0.755593	0.000000

The final model included:

- Param 1
- Param 2

```
# fit_log.summary()
```

The fitted model produced:

- $R^2 = 0.818$
- Adjusted $R^2 = 0.812$
- F-statistic = 141.6
- $p < 0.001$

indicating strong overall explanatory power.

Important Continuous Predictors

Variable	Coefficient
OverTime	2.07
Business Travel	2.02/1.05
Marital Status	1.60

OverTime

Employees working overtime have roughly 8 times the odds of attrition compared with employees who do not, holding the other variables constant:

$$e^{2.07} \approx 7.95$$

Business Travel

Employees traveling frequently have roughly 7.5 times the odds of attrition compared to employees who do not travel, holding the other variables constant:

$$e^{2.02} \approx 7.5$$

Employees traveling rarely also have roughly 2.9 times the odds of attrition compared to employees who do not travel, holding the other variables constant:

$$e^{1.05} \approx 2.9$$

Marital Status

Single employees have roughly 5 times the odds of attrition compared to divorced employees:

$$e^{1.60} \approx 5$$

Evaluate

Residual Analysis

Residual diagnostics indicated:

- No severe violations of linearity.
- Acceptable residual symmetry.
- Some heavy tails remain.
- No extreme multicollinearity concerns.

The largest VIF values were associated with popular truck categories but remained below commonly cited thresholds requiring corrective action.

Limitations

Several limitations should be noted:

- No MSRP information was available.
- Trim level information was not incorporated.
- Accident history was unavailable.
- Maintenance records were unavailable.
- Auction data may not represent all vehicle markets.

These limitations likely explain a portion of the unexplained variance.

Predict

Analysis goes here:

Business Implications

The model suggests:

- 1.

Recommendations

-

Future Work

Future improvements could include:

- 1.

Conclusion

Our conclusion is: